

DEPLOY

Deploy gratuito — PepMem-AI

Guia para publicar o dashboard e compartilhar com colaboradores **sem custo**.

O que vai online

Componente	Recomendação	URL para colaboradores
Dashboard Streamlit	Sim (principal)	Uma URL — predição, ranking, datasets
API FastAPI	Opcional	Só se alguém precisar integrar via HTTP

O dashboard já cobre o uso típico. A API pode ficar local ou ir para o Render (tier free).

Tamanho do deploy: ~15 MB (código + data/processed/). Os dados brutos OPM (data/raw/) **não** precisam subir.

Primeira predição de sequência nova: o modelo ESM-2 (~150 MB) baixa do Hugging Face na hora; pode levar 1–3 min na primeira vez.

Deploy rápido (recomendado)

Passo A — GitHub

1. Crie um repo **vazio** em github.com/new (ex.: PepMem-AI, público).
2. No terminal:

```
cd /home/gab/Documentos/PepMem-AI
./scripts/deploy_github.sh SEU_USUARIO_GITHUB PepMem-AI
git push -u origin main
```

O commit inicial já está pronto localmente (main, ~8 MB de data/processed/).

Passo B — Hugging Face Space

1. Crie token em huggingface.co/settings/tokens (permissão **write**).
2. Login e deploy:

```
hf auth login
python scripts/deploy_hf_space.py SEU_USUARIO_HF --space pepmem-ai
```

URL final: https://huggingface.co/spaces/SEU_USUARIO_HF/pepmem-ai

O script monta `.deploy_hf/` (~8 MB), usa PyTorch **CPU** (`requirements-space.txt`) e copia `deploy/README_HF.md` como card do Space (frontmatter YAML — **não** substituir por `docs/DEPLOY.md`).

Para só testar o pacote localmente sem upload:

```
python scripts/deploy_hf_space.py testuser --build-only
ls .deploy_hf/
```

Opção 1 — Hugging Face Spaces (manual)

Melhor para projetos de ML: CPU gratuita, link estável, fácil de compartilhar.

Passos

1. Crie conta em huggingface.co.
2. Suba o projeto no GitHub (público):

```
cd /home/gab/Documentos/PepMem-AI
git init
git add .
git commit -m "PepMem-AI PoC para deploy"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/PepMem-AI.git
git push -u origin main
```

Confirme que `data/processed/` entrou no commit (modelos + embeddings).

3. Em huggingface.co/new-space:
 - o **Space name:** `pepmem-ai` (ou outro)
 - o **License:** Apache 2.0 ou MIT
 - o **SDK:** Streamlit
 - o **Hardware:** CPU basic (free)
 - o **Visibility:** Public
4. Conecte o repositório GitHub **ou** faça push direto:

```
git clone https://huggingface.co/spaces/SEU_USUARIO/pepmem-ai
cd pepmem-ai
# copie os arquivos do projeto (exceto data/raw)
cp -r ../PepMem-
```

```
AI/{pepmem,dashboard,api,data,scripts,requirements.txt,.streamlit
} .
git add .
git commit -m "Deploy PepMem-AI dashboard"
git push
```

5. Edite o README.md do Space e coloque **no topo** (substitua o README longo por um curto, ou use só este bloco):

```
---
title: PepMem-AI
emoji: 🧬
colorFrom: blue
colorTo: green
sdk: streamlit
sdk_version: "1.31.0"
app_file: dashboard/app.py
pinned: false
---
# PepMem-AI
PoC – predição peptídeo-membrana (InovAI Lab / UFRN).
```

6. Aguarde o build (10–20 min na primeira vez por causa do PyTorch). URL final: https://huggingface.co/spaces/SEU_USUARIO/pepmem-ai
-

Opção 2 — Streamlit Community Cloud (como o Gertec)

Mesmo fluxo do PoC Gertec: repo no GitHub → **New app** no share.streamlit.io.

1. Conta em share.streamlit.io (login com GitHub).
2. **New app** → conecte o GitHub e escolha:
 - o **Repository:** gbmotta/PepMem-AI
 - o **Branch:** main
 - o **Main file path:** dashboard/app.py
 - o **App URL:** pepmem-ai (ou outro nome livre)
 - o **Python version:** 3.11 ou 3.12 (Advanced settings)
3. Deploy. URL pública: <https://pepmem-ai.streamlit.app> (ou o nome escolhido).

O requirements.txt do Cloud é **leve (sem PyTorch)** — usa RF baseline + PMI.

Multimodal (ESM-2) continua no Space HF:

<https://huggingface.co/spaces/gbmotta/pepmem-ai>
Após o push: **Reboot app** no Manage app.

Se o build falhar ainda assim: cole o log do terminal (Manage app) ou use o Space HF / Render.

Opção 3 — Render (Docker, free tier)

O app “dorme” após ~15 min sem uso; acorda em ~1 min no primeiro acesso.

1. Conta em render.com.
 2. **New → Blueprint** (ou Web Service) e aponte para o repo GitHub.
 3. O arquivo `render.yaml` + `Dockerfile` já estão prontos na raiz.
 4. URL: `https://pepmem-ai.onrender.com` (nome escolhido no painel).
-

API FastAPI (opcional)

Para colaboradores que queiram curl/integração:

```
# Local
PYTHONPATH=. uvicorn api.main:app --host 0.0.0.0 --port 8001
```

No Render, crie um **segundo** Web Service com:

```
# Dockerfile.api
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY pepmem/ api/ data/processed/ ./
ENV PYTHONPATH=/app
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "10000"]
```

Porta 10000 é a padrão do Render free.

Checklist antes de publicar

- ☐ `data/processed/models/*.joblib` commitados
- ☐ `data/processed/embeddings/esm2_all.npz` commitado

- ☐ data/processed/*.parquet commitados
 - ☐ .gitignore exclui data/raw/ (opcional, economiza ~19 MB)
 - ☐ Sem .env ou chaves no repositório
 - ☐ Teste local: `streamlit run dashboard/app.py`
-

O que enviar aos colaboradores

Mensagem sugerida:

PepMem-AI (PoC) — dashboard online: [URL]

- Aba **Predição**: sequência + membrana-alvo → PMI e probabilidade de alta atividade
- Aba **Ranking**: compara o peptídeo em várias membranas
- Aba **Datasets**: resumo dos dados integrados (OPM, APD, StigA6/16)

Sequências de exemplo: StigA6 FFSLIPKLVKGLISAFK, StigA16
FFKLIPKLVKGLISAFK

Repo: [GitHub URL] · Documentação: README.md

Problemas comuns

Sintoma	Solução
Build timeout	HF Spaces ou Render; reduzir deps (já usa ESM-2 pequeno t6_8M)
Out of memory	Space CPU basic tem ~16 GB — suficiente; evite use_embeddings=False + recalcular tudo
Sequência nova lenta	Normal na 1ª vez (download ESM-2); depois fica em cache
App dormindo (Render)	Free tier; avise colaboradores que o 1º clique demora